

WRITE UP CAPTURE THE FLAG

POLRI CTF



Tim Stalemater:

Ananditya Narendratama Widagdo

Ardan Desta Sawantino

DAFTAR ISI

WEB EXPLOITATION	3
Pointers	3
MISCELLANEOUS	9
Kunci Rahasia (No AI).....	9
REVERSE ENGINEERING	10
vibe-check	10
cocoon	12
Side Man Coding	14

WEB EXPLOITATION

Pointers

WEB DYNAMIC **pointers** ×

50 pts 60 solves

 First blood: **CSI - Nyolder aja ngapain ngoding**

Pointers adalah layanan tautan mungil yang di-host sendiri: tempel sebuah URL, server mengambilnya dan membuat kartu pratinjau, lalu kamu dapat tautan pendek. Tanpa login, tanpa captcha, semuanya HTTP/JSON biasa. Tentu saja si pengambil pratinjau menolak menyentuh jaringan privat kami. Tentu saja.

Akses: <http://18.143.187.232:8083/>

 **Already solved**

Diberikan akses ke websitenya, gas kita buka

The screenshot shows the Pointers website interface. At the top, there's a logo and the tagline "every link needs a pointer". The main section is titled "Preview a link" with the instruction "paste any http(s) URL — our server fetches it and builds a preview card for you." Below this is a text input field containing "https://example.com" and a blue "Preview" button. A red error message "not a valid URL" is displayed below the input field. The second section is titled "Your pointers" with the instruction "short, shareable redirects." Below this is a table with four columns: SHORT, TITLE, TARGET, and HITS.

SHORT	TITLE	TARGET	HITS
/go/pg	http://127.0.0.1.nip.io:9000/flag	http://127.0.0.1.nip.io:9000/flag	11
/go/pf	http://localhost:9000/admin	http://localhost:9000/admin	2
/go/pe	http://127.0.0.1:9000/flag	http://127.0.0.1:9000/flag	17
/go/pd	https://example.com	https://example.com	2

Sepertinya ini adalah website yang identik dengan command curl yang ada di linux, di mana jika kita memasukkan sebuah url apapun pada kolom input yang ada di website ini, maka akan mengembalikan response dari url yang kita input tadi.

The screenshot shows the Pointers website interface with a successful preview. The "Preview a link" section has the input field containing "https://www.google.com/" and the "Preview" button. Below the input field, the response is displayed, starting with "200 - TEXT/HTML; CHARSET=ISO-8859-1" and "Google". The response body shows the HTML content of the Google homepage, including the meta charset and the main content area.

Saya tes dengan memasukkan url google.com, dan benar, website ini mengembalikan response yang hamper mirip jika kita gunakan curl google.com. Mari kita baca source codenya.

```

45 </div>
46 <div class="foot">Pointers · previews are generated server-side · v1.3</div>
47
48 <script>
49 const $ = (s) => document.querySelector(s);
50 async function preview() {
51   const url = $("#url").value.trim();
52   $("#out").innerHTML = '<div class="notice">fetching...</div>';
53   try {
54     const r = await fetch("/api/preview", { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ url }) });
55     const j = await r.json();
56     if (!j.ok) { $("#out").innerHTML = '<div class="notice err">' + (j.error || "failed") + '</div>'; return; }
57     $("#out").innerHTML =
58       '<div class="preview"><div class="pv-meta">' + (j.status || "") + ' · ' + (j.contentType || "") + '</div>' +
59       '<div class="pv-title">' + (j.title ? escapeHtml(j.title) : "(no title)") + '</div>' +
60       '<div class="pv-desc">' + escapeHtml(j.description || "") + '</div>' +
61       '<div class="pv-url">' + escapeHtml(j.finalUrl || url) + '</div>' +
62       '<pre class="pv-raw">' + escapeHtml(j.snippet || "") + '</pre></div>';
63   } catch (e) { $("#out").innerHTML = '<div class="notice err">' + e + '</div>'; }
64 }
65 function escapeHtml(s){return String(s).replace(/[&lt;"/g,c=>({'&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', '"': '&quot;'}[c])));}
66 $("#go").onclick = preview;
67 $("#url").addEventListener("keydown", (e) => { if (e.key === "Enter") preview(); });
68 </script>

```

Code ini sangat mencurigakan, apalagi kalimat ‘previews are generated server-side’, kode js ini di sisi client hanya mengirim url ke endpoint /api/preview di server, lalu server atau backend melakukan fetch ke url yang diinput client dan mengembalikan responsnya seperti title, description, status, content-type.

Kode ini memiliki kerentanan SSRF (Server Side Request Forgery)

- Karena server yang melakukan fetch (bukan browser client), server punya jaringan internal yang mungkin berbeda dengan client, server bisa mengakses api tersembunyi, mengakses jaringan local
- Field snippet dan FinalUrl menandakan server membaca isi respons dan mengembalikan sebagian respons tersebut ke bagian client, ini penting karena kalau server bisa diarahkan ke endpoint internal, kita bisa membaca isi respons endpoint internal tersebut.

System

Frontend fetches happen through the internal **control-plane** at <http://127.0.0.1:9000> (loopback only — not exposed). Origin fetches are filtered to keep the previewer off our private network.

Kita bisa lihat di bagian paling bawah web, terdapat pesan ini, dapat diketahui bahwa:

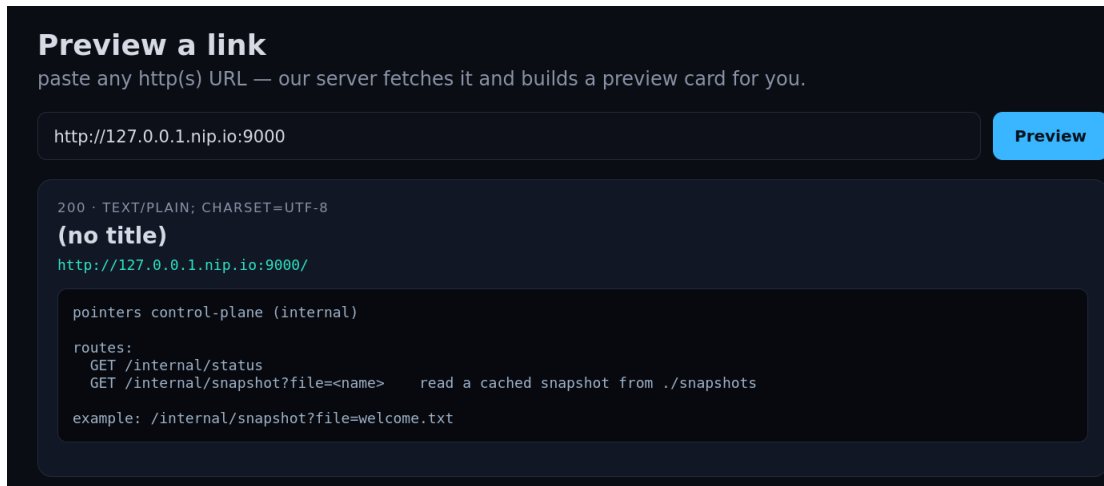
- Internal server berada di <http://127.0.0.1:9000>
- ‘Origin fetches are filtered to keep the previewer off our private network’ kalimat ini menandakan bahwa jika alamat internal server itu terfilter dan tidak bisa kita dapatkan responsnya

Pernyataan ‘Origin fetches are filtered to keep the previewer off our private network’ mengarahkan kita untuk menguji apakah filter tersebut bisa dilewati (bypass).

Kita bisa menggunakan mekanisme resolusi DNS untuk mengarahkannya ke ip internal, ada domain public yang memang didesain untuk merefleksikan query dns ke ip privat tertentu:

- nip.io > 127.0.0.1.nip.io:9000 akan mentranslasikan ke 127.0.0.1:9000

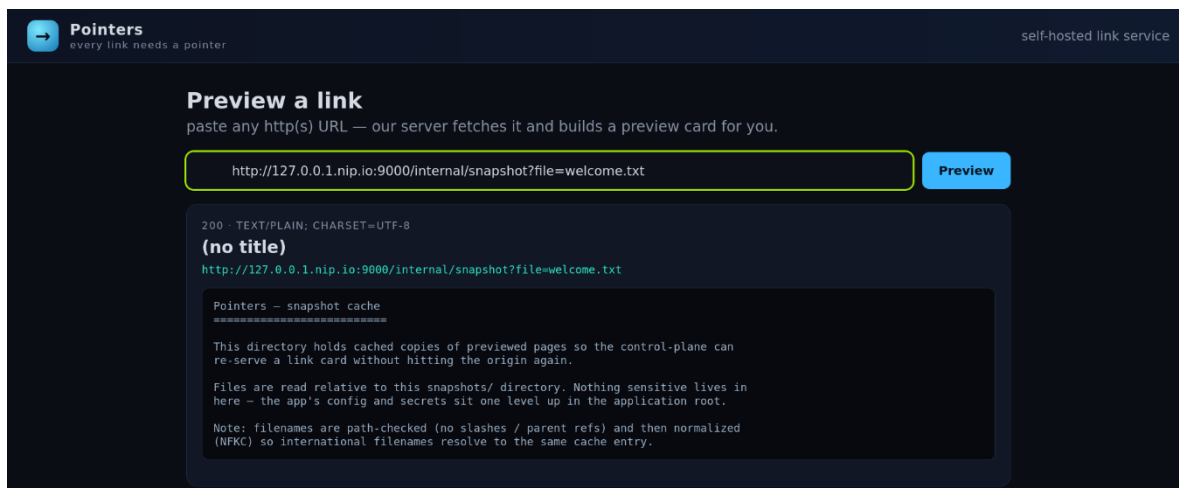
Coba masukkan `http://127.0.0.1.nip.io:9000` ke web tadi



Sepertinya kita telah berhasil melakukan SSRF, dan kita menemukan sebuah endpoint yang menarik yaitu :

- `/internal/snapshot?file=name<name>`

Endpoint tersebut sepertinya bisa membaca sebuah file, sudah dicontohkan juga pada bagian example. Coba lakukan apa yang ada di example.



Sepertinya endpoint `/internal/snapshot?file=` rentang terhadap path traversal, terbukti bahwa tanda `/` dan `,` itu terfilter, cara kerjanya adalah filter mengecek `/` dan `..`, setelah itu baru melakukan normalisasi NFKC, urutan ini adalah celah yang dikenal sebagai Unicode Normalization Bypass, dan `flag/secret` ada di atas 1 level direktori `snapshot`.

Karena karakter ‘/’ dan ‘.’ Terfilter, kita bisa menencode karakter “/” dan “.” tersebut agar bisa membypass filternya.

Yang kita butuhkan adalah itu karakter Unicode look-alike yang bukan karakter “/” dan “.” pada umumnya, tapi setelah dinormalisasi NFKC berubah menjadi “/” dan “.” biasa.

Caranya adalah :

Encode Karakter Unicode ke UTF-8 bytes

- / (U+FFOF, FULLWITDH SOLIDUS) > UTF-8: EF BC 8F
- . (U+FFOE, FULLWITDH FULL STOP) > UTF-8: EF BC 8E

Lalu kita lakukan persent encode setiap byte

- / > %EF%BC%8F
- . > %EF%BC%8E

Secret tersembunyi di atas direktori snapshots, jadi payload sebelum encode adalah :

```
../flag.txt
```

Setelah encode adalah :

```
%EF%BC%8E%EF%BC%8E%EF%BC%8Fflag.txt
```

Payload lengkapnya adalah :

```
http://127.0.0.1.nip.io:9000/internal/snapshot?file=%EF%BC%8E%EF%BC%8E%EF%BC%8Fflag.txt
```

Gas kita masukkan payload lengkap tersebut ke parameter input yang ada!

**Pointers**
every link needs a pointer

self-hosted link service

Preview a link

paste any http(s) URL — our server fetches it and builds a preview card for you.

Preview

200 · TEXT/PLAIN; CHARSET=UTF-8

(no title)

http://127.0.0.1.nip.io:9000/internal/snapshot?file=%EF%BC%8E%EF%BC%8E%EF%BC%8Fflag.txt

polriCTF26{fullwidth_d0ts_p01nt_str41ght_1nt0_th3_c0ntr0l_pl4n3}

Your pointers

short, shareable redirects.

SHORT	TITLE	TARGET	HITS
/go/p2r	kv1	http://127.0.0.1:9000/flag	1

Flag ditemukan!

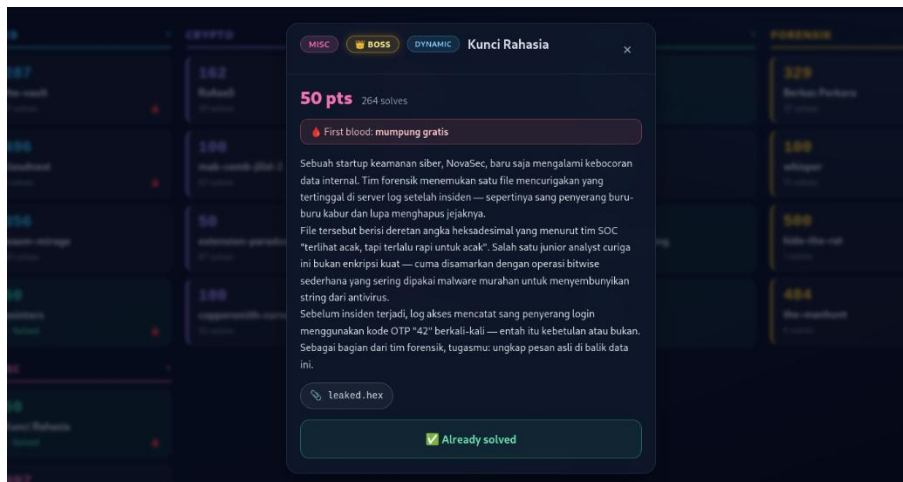
polriCTF26{fullwidth_d0ts_p01nt_str41ght_1nt0_th3_c0ntr0l_pl4n3}

[Link Chat AI](#)

MISCELLANEOUS

Kunci Rahasia (No AI)

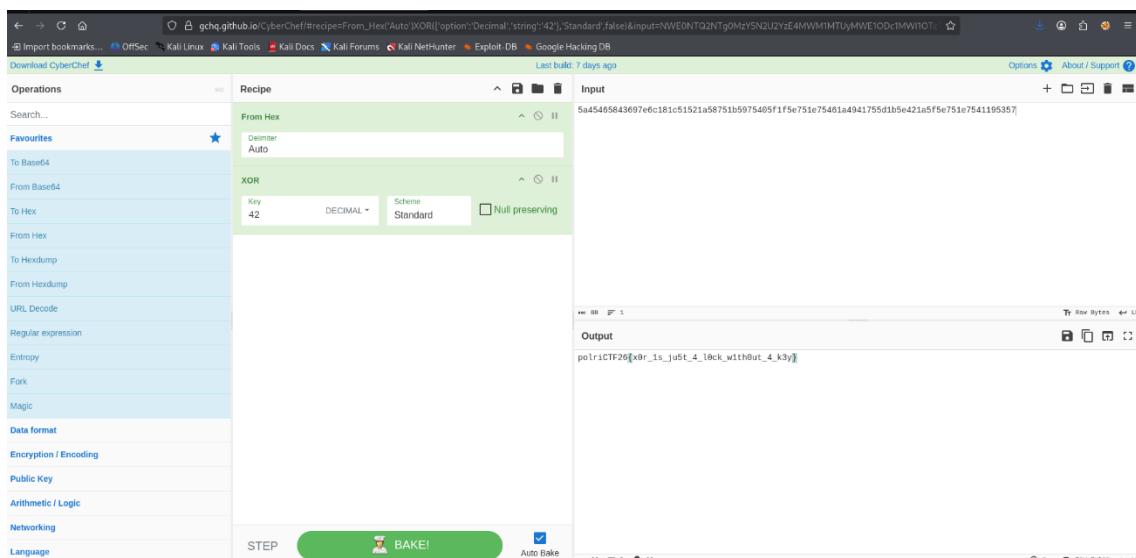
TL;DR: mengubah data hex ke bentuk byte lalu di-XOR kan dengan key 42



Tools Used: cat, CyberChef

Kita diberikan deretan angka hexadecimal, deskripsi challenge mengarah pada operasi bitwise yang sering digunakan malware murahan atau biasa dipakai di CTF dan angka '42' sebagai sebuah OTP yang digunakan pelaku.

Hint tersebut mengarah pada operasi bitwise XOR dengan key = 42. Saya menggunakan tools online CyberChef untuk mendapatkan flagnya dengan urutan decode hexadecimal terlebih dahulu dilanjutkan dengan XOR (key = 42).

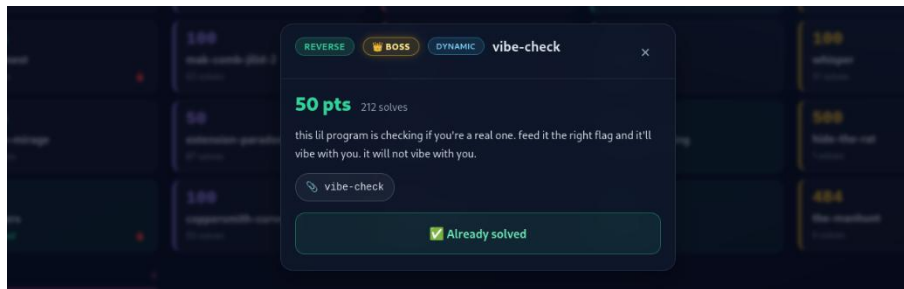


FLAG: polriCTF26{x0r_1s_ju5t_4_l0ck_w1th0ut_4_k3y}

REVERSE ENGINEERING

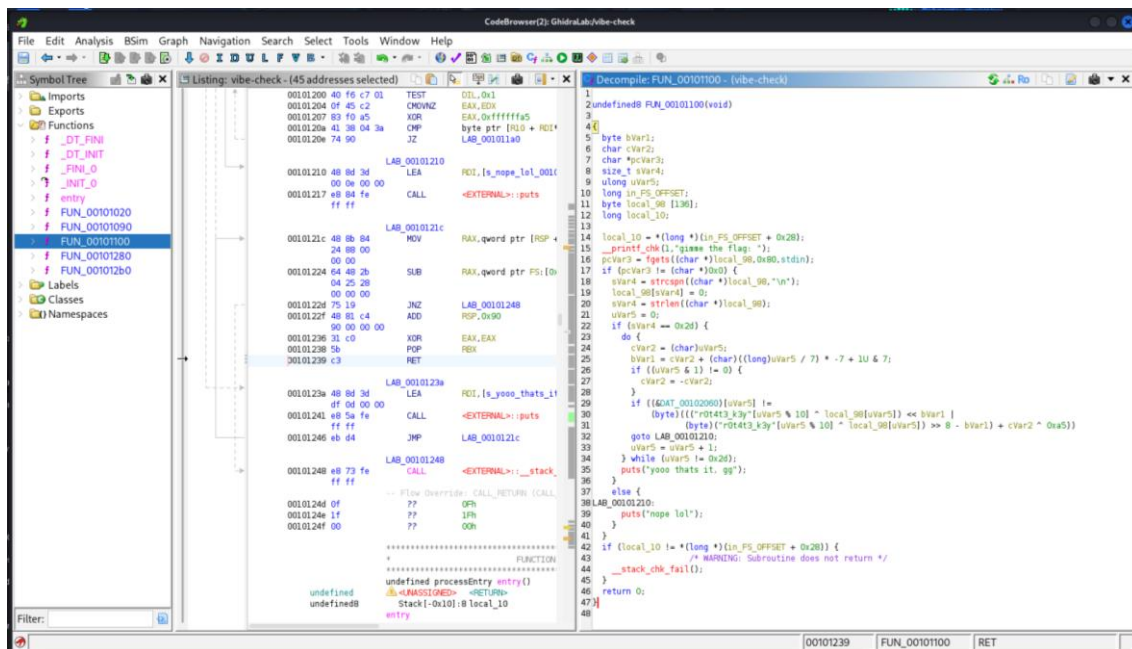
vibe-check

TL;DR: decompile fungsi pemeriksaan flag di Ghidra lalu melakukan reverse operasi matematika & operasi bitwise untuk mendapatkan flag.



Tools Used: Ghidra, python3

Jenis file kali ini adalah sebuah biner ELF 64-bit terkompilasi, langsung saja kita buka filenya di Ghidra untuk melihat detail fungsinya.



Program mengecek input user sepanjang 45 character dan membandingkannya dengan array `DAT\_00102060`.

Kurang lebih pengecekannya seperti ini:

1. XOR dengan key `r0t4t3\_k3y`.
2. Rotate Left sebanyak `bVar1` bit.
3. Penjumlahan dengan variable `cVar2`.

4. XOR dengan key `0xa5`.

Kita akan mengambil nilai dari array `DAT\_00102060` lalu meminta AI membalikan seluruh operasi secara berurutan dari tambah menjadi kurang, kiri menjadi kanan, XOR mah sama aja.

1. XOR dengan key `0xa5`.

2. Pengurangan dengan variable `cVar2`.

3. Rotate Right sebanyak `bVar1` bit.

4. XOR dengan key `r0t4t3\_k3y`.

History Chat AI

```
Reversing > polrivibe.py >...
1 def ROR8(val, shift):
2     shift &= 7
3     return ((val >> shift) | (val << (8 - shift))) & 0xFF
4
5 def solve_from_hex(hex_string):
6     hex_string = hex_string.strip().replace(" ", "")
7     dat_bytes = bytes.fromhex(hex_string)
8
9     key = b"r0t4t3_k3y"
10    flag = []
11
12    for i in range(len(dat_bytes)):
13        target = dat_bytes[i]
14        bVar1 = ((i % 7) + 1) & 7
15        cVar2 = i & 0xFF
16        if (i & 1) != 0:
17            cVar2 = (-cVar2) & 0xFF
18
19        step1 = target ^ 0xA5
20        step2 = (step1 - cVar2) & 0xFF
21        step3 = ROR8(step2, bVar1)
22
23        original_char = step3 ^ key[i % 10]
24        flag.append(chr(original_char))
25
26    return "".join(flag)
27
28 hex_data = "a1d967c402b22ef6a9d43ff8a8d0e707ec019fd39236f3e12d0acc5a19aaf5b3a6906010a1102ebfd6a5934bd1"
29
30 print(solve_from_hex(hex_data))
31
```

Langsung saja kita jalankan scriptnya.

```
(sphinx@stalemater) - [~/lab/POLRICTF26/Reverse/vibe-check-]
$ nano polrivibe.py

(sphinx@stalemater) - [~/lab/POLRICTF26/Reverse/vibe-check-]
$ cat polrivibe.py
def ROR8(val, shift):
    shift &= 7
    return ((val >> shift) | (val << (8 - shift))) & 0xFF

def solve_from_hex(hex_string):
    hex_string = hex_string.strip().replace(" ", "")
    dat_bytes = bytes.fromhex(hex_string)

    key = b"r0t4t3_k3y"
    flag = []

    for i in range(len(dat_bytes)):
        target = dat_bytes[i]
        bVar1 = ((i % 7) + 1) & 7
        cVar2 = i & 0xFF
        if (i & 1) != 0:
            cVar2 = (-cVar2) & 0xFF

        step1 = target ^ 0xA5
        step2 = (step1 - cVar2) & 0xFF
        step3 = ROR8(step2, bVar1)

        original_char = step3 ^ key[i % 10]
        flag.append(chr(original_char))

    return "".join(flag)

hex_data = "a1d967c402b22ef6a9d43ff8a8d0e707ec019fd39236f3e12d0acc5a19aaf5b3a6906010a1102ebfd6a5934bd1"

print(solve_from_hex(hex_data))

(sphinx@stalemater) - [~/lab/POLRICTF26/Reverse/vibe-check-]
$ python3 polrivibe.py
polriCTF26{sp1n_x0r_4dd_sub_cr4ckm3_g0es_brr}
```

FLAG: polriCTF26{sp1n_x0r_4dd_sub_cr4ckm3_g0es_brr}

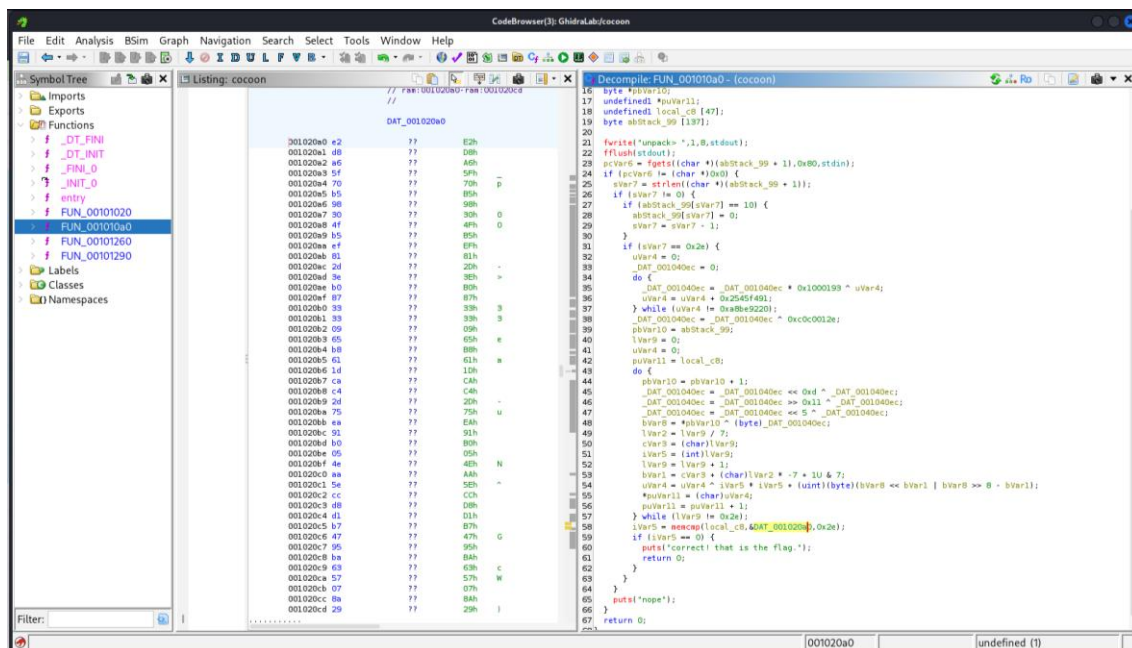
cocoon

TL;DR: Membalik PRNG Xorshift32, rotasi bit, dan XOR pada array `DAT\_001020a0`.



Tools Used: Ghidra, python3

Sama seperti sebelumnya, jenis file disini adalah ELF 64-bit, langsung buka Ghidra untuk melihat detail fungsi-nya.



Tab Tengah Adalah nilai array `DAT\_001020a0` Kurang lebih seperti ini urutan pengecekannya:

1. Inisialisasi status PRNG `\_DAT\_001040ec` dari loop dan XOR.
2. Setiap character input di-XOR kan dengan byte PRNG Xorshift32.
3. Hasil XOR di Rotate Left sebanyak `bVar1` bit.

4. Nilai rotate ditambah i^2 lalu di-XOR ke alumulator `uVar4`.

5. Hasil byte `uVar4` dibandingkan dengan array `` (46 byte).

Kita akan mengambil raw data dari array `DAT\_001020a0` dan meminta AI untuk membalikan proses dan membuat script Python untuk mendapatkan flagnya.

Proses pembalikannya kira-kira seperti ini:

1. Menghitung nilai awal PRNG `\_DAT\_001040ec` dengan unsigned 32-bit integer
2. Membuat byte dari `DAT\_001020a0`, lalu mengisolasi $(i^2 + R)$ dari nilai akmulator sebelumnya.
3. Membalikan Rotate Left dengan Rotate Right sebanyak `bVar1` bit.
4. XOR Kembali nilai dengan byte PRNG Xorshift32 untuk mendapatkan flag.

History Chat AI

```
Reversing > policooon.py > ...
1 def ROR8(val, shift):
2     shift &= 7
3     return ((val >> shift) | (val << (8 - shift))) & 0xFF
4
5 def solve_dat20a0(hex_string):
6     hex_string = hex_string.strip().replace(" ", "")
7     dat_bytes = bytes.fromhex(hex_string)
8
9     uVar4 = 0
10    dat_40ec = 0
11    while True:
12        dat_40ec = ((dat_40ec * 0x1000193) & 0xFFFFFFFF) ^ uVar4
13        uVar4 = (uVar4 + 0x2545f491) & 0xFFFFFFFF
14        if uVar4 == 0xa8be9220:
15            break
16
17    dat_40ec = dat_40ec ^ 0xc0c012e
18
19    flag = []
20    prev_uVar4 = 0
21
22    for i in range(len(dat_bytes)):
23        target_byte = dat_bytes[i]
24
25        dat_40ec = (dat_40ec ^ ((dat_40ec << 13) & 0xFFFFFFFF))
26        dat_40ec = (dat_40ec ^ (dat_40ec >> 17))
27        dat_40ec = (dat_40ec ^ ((dat_40ec << 5) & 0xFFFFFFFF))
28
29        prng_byte = dat_40ec & 0xFF
30        bVar1 = ((i % 7) + 1) & 7
31
32        term = ((prev_uVar4 ^ target_byte) - (i * 1)) & 0xFF
33
34        prev_uVar4 = prev_uVar4 ^ (i * 1 + term)
35        bVar8 = ROR8(term, bVar1)
36        original_char = bVar8 ^ prng_byte
37
38        flag.append(chr(original_char))
39
40    return ''.join(flag)
41
42 hex_data = "MASUKKAN_STRING_HEX_KAMU_DI_SINI"
43 print(solve_dat20a0(hex_data))
44
```

Langsung saja jalankan scriptnya dengan nilai variable hex_data dari array `DAT\_001020a0`.

```

sphinx@stalemater: ~/Lab/POLRICTF26/Reverse/cocoon-
$ nano polricocoon.py
sphinx@stalemater: ~/Lab/POLRICTF26/Reverse/cocoon-
$ cat polricocoon.py
def ROT8(val, shift):
    shift = 7
    return ((val >> shift) | (val << (8 - shift))) & 0xFF

def solve_dat20a0(hex_string):
    hex_string = hex_string.strip().replace(" ", "")
    dat_bytes = bytes.fromhex(hex_string)

    uVar4 = 0
    dat_40ec = 0
    while True:
        dat_40ec = ((dat_40ec + 0x1000103) & 0xFFFFFFFF) ^ uVar4
        uVar4 = (uVar4 + 0x2545f491) & 0xFFFFFFFF
        if uVar4 == 0xabbe9220:
            break

    dat_40ec = dat_40ec ^ 0xc0c0012e

    flag = []
    prev_uVar4 = 0

    for i in range(len(dat_bytes)):
        target_byte = dat_bytes[i]

        dat_40ec = (dat_40ec ^ ((dat_40ec << 13) & 0xFFFFFFFF))
        dat_40ec = (dat_40ec ^ (dat_40ec >> 17))
        dat_40ec = (dat_40ec ^ ((dat_40ec << 5) & 0xFFFFFFFF))

        prng_byte = dat_40ec & 0xFF
        bVar1 = ((i % 7) + 1) & 7

        term = ((prev_uVar4 ^ target_byte) - (i * i)) & 0xFF

        prev_uVar4 = prev_uVar4 ^ (i * i + term)
        bVar0 = ROT8(term, bVar1)
        original_char = bVar0 ^ prng_byte

        flag.append(chr(original_char))

    return "".join(flag)

hex_data = "e2d5a65f78b598304fb5ef812d3eb087333096508611dcac42d75ea91b0054eaa5eccd8d1b74795ba6357078a29"
print(solve_dat20a0(hex_data))

sphinx@stalemater: ~/Lab/POLRICTF26/Reverse/cocoon-
$ python3 polricocoon.py
polriCTF26{p331_th3_l4y3rs_x0r_r0t_qu4d_ch41n}

```

FLAG: polriCTF26{p331_th3_l4y3rs_x0r_r0t_qu4d_ch41n}

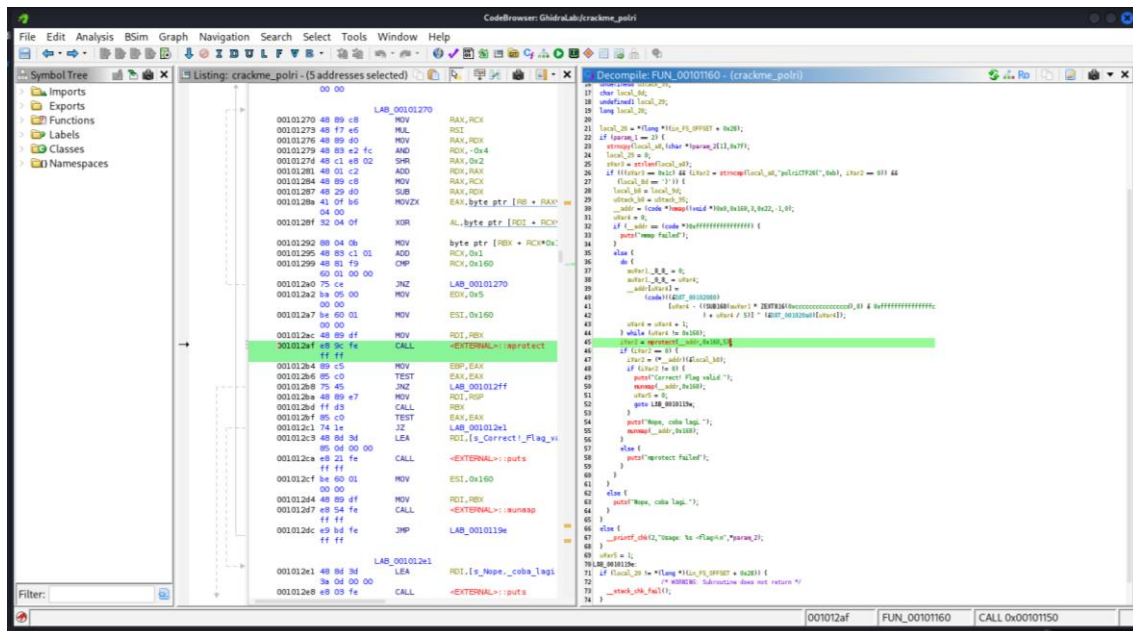
Side Man Coding

TL;DR: Memakai GDB untuk break tepat saat code yang dibuat secara dinamis (Self-Modifying Code) dideskripsi di memori, lalu membalikkan enkripsinya.



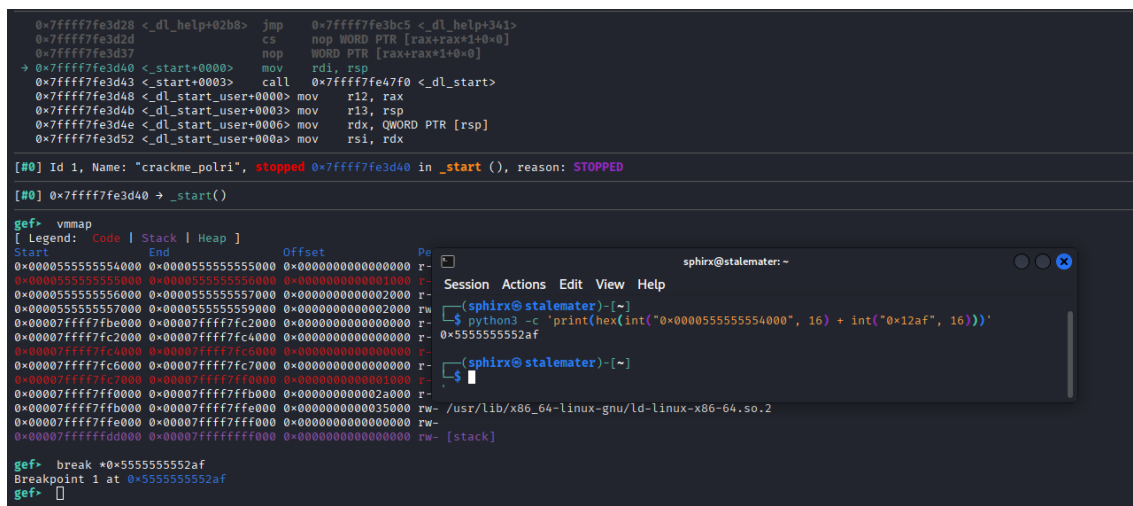
Tools Used: Ghidra, python3

Sebuah ELF 64-bit yang memakai Teknik Self-Modifying Code (SMC), karena biner ini membuat kodenya sendiri saat dijalankan, kita tidak bisa hanya menggunakan tools Static (Ghidra/IDA) dan memerlukan tools Dynamic.



Kali ini saya akan menggunakan tools GDB (with plugin GEF) untuk melakukan analisis dynamic, pertama-tama kita akan memasang breakpoint pada fungsi `mprotect` yang berfungsi sebagai pembuka izin proteksi memori dan mengeksekusi program sampai tepat sebelum instruksi code dynamic `call rbx`. Saya mengambil offset fungsi `mprotect` yang akan berguna nantinya dari Ghidra sekaligus melihat bentuk statisnya dulu.

Langsung saja kita buka tools GDB (gef) dan mencari base addressnya dengan perintah `vmmap`. Jangan lupa eksekusi program terlebih dahulu menggunakan `starti`.



Terlihat pada gambar diatas saya menjumlahkan base address dengan offset dari fungsi `mprotect` dan menjadikan hasil alamatnya sebagai breakpoint.

Lanjut, kita akan memasukan perintah `run "polriCTF26{AAAAAAAAAAAAAAAAAAAA}"` untuk mengeksekusi program sekaligus mengirimkan input.

```

0x555555552a2      mov     edi, 0x5
0x555555552a7      mov     esi, 0x100
0x555555552ab      mov     edi, 0x0
0x555555552af      call   0x5555555515b <mprotect@plt>
0x55555555150 <mprotect@plt+0x000> endbr64
0x55555555154 <mprotect@plt+0x004> jmp     QWORD PTR [rip+0x2e76] # 0x555555557fd0 <mprotect@got.plt>
0x5555555515a <mprotect@plt+0x006> nopl   WORD PTR [rax+rax*1+0x0]
0x55555555160      endbr64
0x55555555164      push    rbp
0x55555555165      push    rdx

mprotect@plt {
  $r10 = 0x00007ffff7fb0000 + 0xd87d948e5894855,
  $rax = 0x0000000000000160,
  $rdx = 0x0000000000000005,
  $rcx = 0x0000000000000160
}

[0] Id 1, Name: "crackme_polri", stopped 0x555555552af in 77 (), reason: BREAKPOINT
[0] 0x555555552af + call 0x5555555515b <mprotect@plt>
[1] 0x7ffff7d04277 + __libc_start_call_main(main=0x55555555160, argc=0x2, argv=0x7ffff7d04b08)
[2] 0x7ffff7d04027 + __libc_start_main_impl(main=0x55555555160, argc=0x2, argv=0x7ffff7d04b08, init=optimized out, fini=optimized out, rtld_fini=optimized out, stack_end=0x7ffff7d04b58)
[3] 0x55555555355 + 0x1
gef-

```

Pastikan titik merah pada pojok kiri atas menunjuk pada fungsi `mprotect`, jika belum maka pakai perintah `nexti` sedikit demi sedikit untuk melanjutkan program.

Malah jadi dokumen tutorial wkwk, ini asli ngetik sendiri yaa, bukan dari AI, soalnya pakai technic Feynman biar diri sendiri hafal.

Setelah ini kita akan menampilkan 100 kode assembly untuk mencaritahu petunjuk flag menggunakan perintah `x/100i \$rbx`. Saya akan menyerahkan ini kepada AI karena saya belum terlalu mendalami Assembly.

```

gef- x/100i $rbx
0x7ffff7d0000: push    rbp
0x7ffff7d0001: mov     rbp, rsp
0x7ffff7d0004: mov     QWORD PTR [rbp-0x20], rdi
0x7ffff7d0008: mov     DWORD PTR [rbp-0x4], 0x0
0x7ffff7d000f: jmp     0x7ffff7d0000
0x7ffff7d0011: mov     eax, DWORD PTR [rbp-0x4]
0x7ffff7d0014: movsxd  rdi, eax
0x7ffff7d0017: mov     rax, QWORD PTR [rbp-0x20]
0x7ffff7d001b: add     rax, rdi
0x7ffff7d001e: movzx   eax, BYTE PTR [rax]
0x7ffff7d0021: mov     BYTE PTR [rbp-0x0], al
0x7ffff7d0024: xor     BYTE PTR [rbp-0x0], 0x5a
0x7ffff7d0028: mov     eax, DWORD PTR [rbp-0x4]
0x7ffff7d002b: mov     rax, eax
0x7ffff7d002d: mov     rax, rax
0x7ffff7d002f: add     rax, eax
0x7ffff7d0031: add     rax, rax
0x7ffff7d0033: add     BYTE PTR [rbp-0x0], al
0x7ffff7d0036: movzx   eax, BYTE PTR [rbp-0x0]
0x7ffff7d003a: lea     rax, [rax*0x0]
0x7ffff7d0041: movzx   eax, BYTE PTR [rbp-0x0]
0x7ffff7d0045: shr     al, 0x6
0x7ffff7d0048: or      rax, rax
0x7ffff7d004a: mov     BYTE PTR [rbp-0x0], al
0x7ffff7d004c: mov     eax, DWORD PTR [rbp-0x4]
0x7ffff7d004e: cqoq
0x7ffff7d0051: movzx   rdi, BYTE PTR [rbp-0x0]
0x7ffff7d0054: mov     BYTE PTR [rbp-0x0+1-0x20], al
0x7ffff7d005a: add     DWORD PTR [rbp-0x4], 0x1
0x7ffff7d005e: cmp     DWORD PTR [rbp-0x4], 0x1
0x7ffff7d0062: jle     0x7ffff7d0011
0x7ffff7d0064: mov     DWORD PTR [rbp-0x0], 0x1
0x7ffff7d0066: movzx   rdi, BYTE PTR [rbp-0x0]
0x7ffff7d006f: cmp     al, 0x24
0x7ffff7d0071: je      0x7ffff7d007a
0x7ffff7d0073: mov     DWORD PTR [rbp-0x0], 0x0
0x7ffff7d007a: movzx   eax, BYTE PTR [rbp-0x0]
0x7ffff7d007e: cmp     al, 0x60
0x7ffff7d0080: je      0x7ffff7d0089
0x7ffff7d0082: mov     DWORD PTR [rbp-0x0], 0x0
0x7ffff7d0085: movzx   eax, BYTE PTR [rbp-0x0]
0x7ffff7d0087: cmp     al, 0x7
0x7ffff7d008f: je      0x7ffff7d0098
0x7ffff7d0091: mov     DWORD PTR [rbp-0x0], 0x0
0x7ffff7d0094: movzx   rdi, BYTE PTR [rbp-0x0]
0x7ffff7d0097: cmp     al, 0x30
0x7ffff7d0099: je      0x7ffff7d00a7
0x7ffff7d009b: mov     DWORD PTR [rbp-0x0], 0x0
0x7ffff7d009d: movzx   eax, BYTE PTR [rbp-0x0]
0x7ffff7d00a0: cmp     al, 0x60
0x7ffff7d00a2: je      0x7ffff7d00b0
0x7ffff7d00a4: mov     DWORD PTR [rbp-0x0], 0x0
0x7ffff7d00a6: movzx   rdi, BYTE PTR [rbp-0x0]

```

AI memberikan sebuah script Python sekaligus array target pembanding dari Kumpulan 100 kode assembly. Dengan algoritma enkripsi seperti ini:

1. $x = \text{input_char} \wedge 0x5a$
2. $x = (x + i * 3) \& 0xff$
3. $x = \text{ROL8}(x, 2)$ (karena $(x * 4) \mid (x \gg 6)$)

History Chat AI


```

Reversing > polriSMC.py > ...
1  target = [
2      0x24, 0x68, 0x7c, 0x38, 0xdd, 0x60, 0x5c, 0x78,
3      0x0a, 0x80, 0xdc, 0x2e, 0x3a, 0xf4, 0x96, 0xa2
4  ]
5
6  def ROR8(val, shift):
7      return ((val >> shift) | (val << (8 - shift))) & 0xFF
8
9  flag = []
10 for i, enc in enumerate(target):
11     x = ROR8(enc, 2)
12     x = (x - (i * 3)) & 0xFF
13     p = x ^ 0x5A
14     flag.append(chr(p))
15
16 print("Flag:", "".join(flag))

```

Saya sesuaikan sedikit agar langsung menampilkan hasil polriCTF26{*}. Langsung saja jalankan scriptnya untuk melihat hasilnya.

```

(sphirx@stalemater)-[~/lab/POLRICTF26/Reverse/Side Man Coding-]
$ nano polriSMC.py

(sphirx@stalemater)-[~/lab/POLRICTF26/Reverse/Side Man Coding-]
$ cat polriSMC.py
target = [
    0x24, 0x68, 0x7c, 0x38, 0xdd, 0x60, 0x5c, 0x78,
    0x0a, 0x80, 0xdc, 0x2e, 0x3a, 0xf4, 0x96, 0xa2
]

def ROR8(val, shift):
    return ((val >> shift) | (val << (8 - shift))) & 0xFF

flag = []
for i, enc in enumerate(target):
    x = ROR8(enc, 2)
    x = (x - (i * 3)) & 0xFF
    p = x ^ 0x5A
    flag.append(chr(p))

print(f"polriCTF26{{{'.join(flag)}}}")

(sphirx@stalemater)-[~/lab/POLRICTF26/Reverse/Side Man Coding-]
$ python3 polriSMC.py
polriCTF26{SMC_1S_S0_C00L!!}

```

FLAG: polriCTF26{SMC_1S_S0_C00L!!}

RIWAYAT PERCAKAPAN AI

[History AI I](#)

[History AI II](#)

[History AI III](#)